+34 691 225 633
Rev. Febrero 2024

Autor: Jefferson David Rodríguez Chivatá

Visualizando otras estructuras de datos: Grafos con NetworkX

Introducción

Bienvenidos a esta guía de profundización en la segunda semana, donde exploraremos y visualizaremos datos en forma de grafos y redes utilizando NetworkX, una herramienta esencial para visualizar y analizar estructuras de datos complejas. En esta sesión, nos enfocaremos en los fundamentos de NetworkX, proporcionando una base sólida para aquellos que se inician en la visualización de estructuras de datos complejas. Exploraremos cómo estructurar los datos de manera efectiva y utilizaremos las capacidades de NetworkX para transformar estos datos en visualizaciones claras y significativas. A lo largo de esta guía, aprenderemos a manejar las funcionalidades básicas de NetworkX, desde la creación de grafos simples hasta la implementación de algoritmos básicos para el análisis de redes. Nuestro objetivo es brindarles una comprensión práctica y accesible de cómo NetworkX puede ser una herramienta invaluable en el campo de la visualización de datos, abriendo la puerta a un análisis más profundo y detallado de relaciones y estructuras complejas. ¡Esperamos que esta introducción siga despertando su interés y curiosidad por la visualización de datos y que encuentren en NetworkX un aliado fundamental en su viaje analítico!

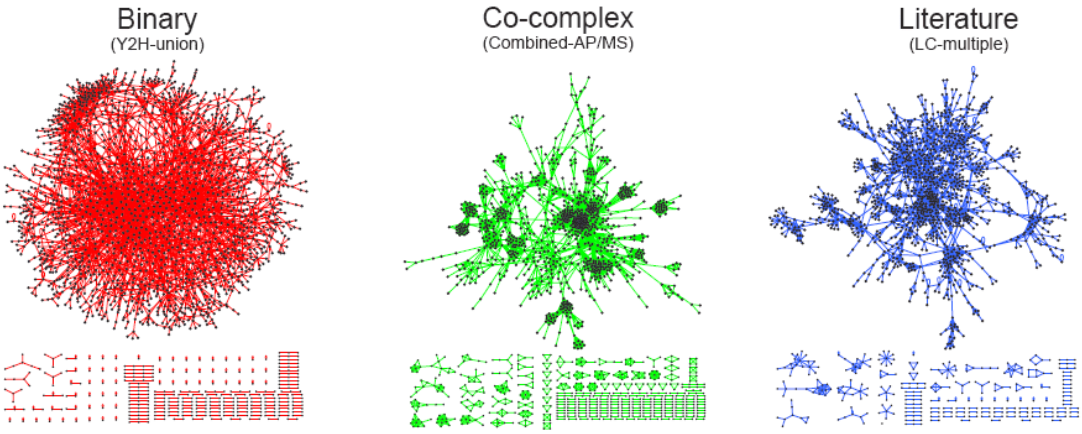
Contenido

Iniciemos nuestra sesión adentrándonos en el concepto de grafos, una estructura fundamental para la visualización de datos. Los grafos son representaciones que nos permiten visualizar las relaciones y conexiones entre distintos elementos, siendo clave en áreas tan diversas como las ciencias sociales, la biología (*ver figura 1*), la informática y más. La importancia de contar con una herramienta de visualización de datos para explorar grafos radica en su capacidad para transformar datos abstractos y complejos en representaciones claras y comprensibles. Con estas herramientas, no solo descubrimos patrones y

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

conexiones ocultas, sino que también facilitamos la interpretación y análisis de las interacciones y estructuras subyacentes. En esta sesión, veremos cómo NetworkX, una biblioteca poderosa y versátil, nos permite desentrañar la complejidad de los grafos y extraer valiosas percepciones de ellos.

Figura 1: High quality binary protein interaction map



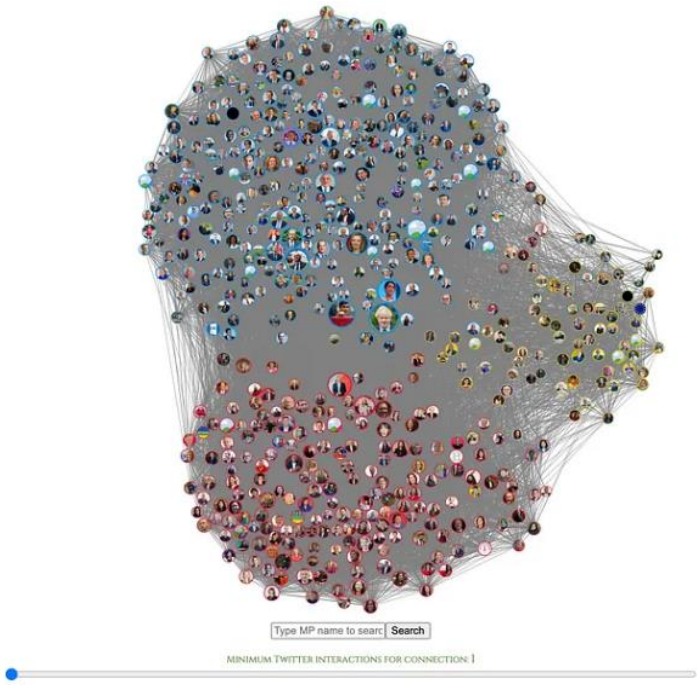
Fuente: Tomado de 'Yeast Interactome Project' por Ye et al, 2008, recuperado de Yeast Interactome Database.

- **NetworkX:**

NetworkX es una biblioteca de Python para analizar, modelar y visualizar redes y grafos complejos. Proporciona herramientas para trabajar con grafos, incluyendo algoritmos para generar grafos aleatorios, estadísticas de grafos, análisis de grafos y visualización de grafos. NetworkX soporta varios tipos de grafos, incluyendo grafos dirigidos y no dirigidos, grafos ponderados y no ponderados, y multigrafos. Es ampliamente utilizado en diversos campos como el análisis de redes sociales, informática, matemáticas y biología. Por ejemplo, la siguiente figura nos muestra una red de relaciones entre los miembros del parlamento británico basado en sus interacciones de Twitter.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 1: High quality binary protein interaction map



Fuente: Tomado de 'Yeast Interactome Project' por Ye et al, 2008, recuperado de Yeast Interactome Database.

Esta gráfica además de dejar en evidencia la importancia de la visualización de grafos está construida con D3.js el framework en el cual se basa NVD3 estudiado en el tema principal de esta semana. Como estos ejemplos mostrados del uso de grafos, hay un sin número de aplicaciones por lo tanto manejar una herramienta como NetworkX es importante para ampliar nuestras habilidades en visualización de datos. Los beneficios que tiene NetworkX son los siguientes:

- **Facilidad de uso:** NetworkX proporciona una interfaz simple e intuitiva para crear y visualizar grafos, haciéndola accesible para usuarios con distintos niveles de experiencia.
- **Personalización:** NetworkX permite la personalización de la apariencia del grafo, incluyendo el tamaño de los nodos, color, etiqueta y estilo de aristas.
- **Escalabilidad:** NetworkX puede manejar grafos grandes con millones de nodos y aristas, lo que lo hace adecuado para el análisis de redes complejas.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Interoperabilidad:** NetworkX puede usarse con otras bibliotecas de Python, como Matplotlib y NumPy, para una visualización y análisis aún más avanzados.
- **Biblioteca de algoritmos:** NetworkX incluye una amplia biblioteca de algoritmos de grafos para tareas como el cálculo del camino más corto, medidas de centralidad y detección de comunidades.
- **Código abierto:** Es un software de código abierto, lo que lo hace disponible gratuitamente para los usuarios y permite el desarrollo y mejora comunitaria.

Guia básica de NetworkX: En esta sesión de lectura vamos a introducir el manejo de NetworkX para la visualización de grafos con un ejemplo práctico. Imagine el siguiente escenario:

- Wayne W. Zachary estudió durante tres años la red social de un club de karate. La red incluía a 34 miembros del club, registrando las interacciones fuera del mismo entre parejas de miembros. Durante el estudio, surgió un conflicto entre el administrador "John A" y el instructor "Mr. Hi", lo que provocó la división del club en dos. La mitad de los miembros creó un nuevo club con Mr. Hi, mientras que los demás encontraron un nuevo instructor o dejaron el karate. Basándose en los datos recopilados, Zachary logró asignar correctamente a casi todos los miembros de los grupos que se unieron tras la división.

Este es un ejemplo muy conocido, nuestro foco principal será visualizar estos datos sin entrar en mucho detalle acerca del objetivo final del trabajo como tal. Así que iniciemos:

1. Cargar datos en NetworkX a partir de un archivo CSV:

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
# Obtener la lista de ejes desde un archivo csv
karate_edgelist = pd.read_csv("https://ona-book.org/data/karate.csv")
# convertir el Dataframe a un grafo de NetworkX
karate = nx.from_pandas_edgelist(karate_edgelist, source = 'from', target = 'to')
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 2: Estructura de los datos de entrada

	from	to
0	Mr Hi	Actor 2
1	Mr Hi	Actor 3
2	Mr Hi	Actor 4
3	Mr Hi	Actor 5
4	Mr Hi	Actor 6
...	...	...
73	Actor 31	Actor 33
74	Actor 31	John A
75	Actor 32	Actor 33
76	Actor 32	John A
77	Actor 33	John A

78 rows × 2 columns

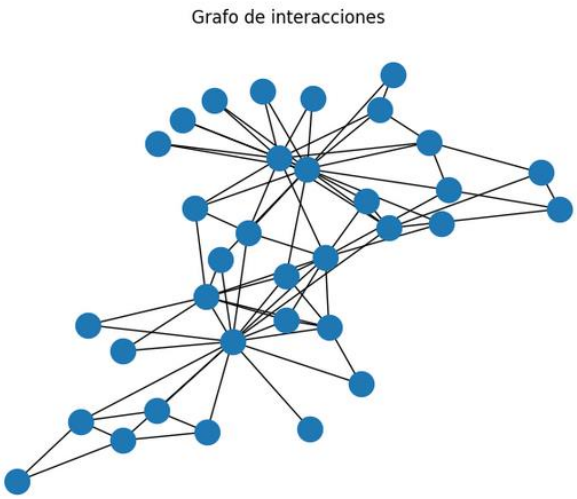
Fuente: Elaboración propia

Como podemos observar la estructura de las interacciones contiene un campo desde donde parte la relación y a que individuo llega, con esto podemos crear el grafo de NetworkX y ver sus propiedades como cuales son los vértices, nodos, si el grafo es dirigido o es un multigrafo, entre otras. Una vez tenemos el objeto grafo, en nuestro caso "*Karate*" basta con llamar a la función *nx.draw* para que nos permita visualizar de manera básica el grafo como sigue usando Matplotlib:

```
# visualizar el grafo usando matplotlib
fig = nx.draw(karate)
# colocar titulo a la figura
plt.title('Grafo de interacciones')
# mostrar el grafo
plt.show()
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Figura 3: Visualización del grafo Karate



Fuente: Elaboración propia

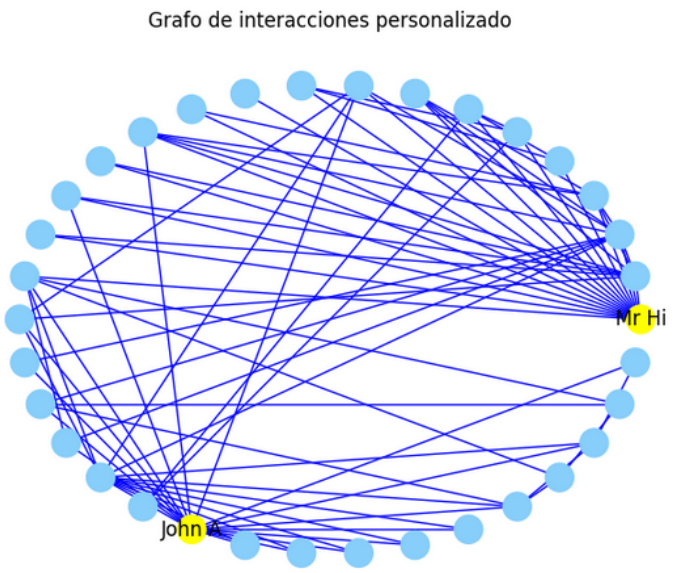
De manera directa tenemos visualizado nuestro grafo de interacciones. Sin embargo, no tiene muchas opciones de personalización, a menos que trabajemos directamente con Matplotlib. Afortunadamente, para efectos prácticos, NetworkX tiene unas funciones que nos permiten agregar personalizaciones (*nx.draw_networkx* y *nx.draw_circular*). Por ejemplo, vamos a agregar color a los nodos y organizar nuestro grafo de manera circular.

```
# recorrer los nodos y solo seleccionar los nombres
# de los nodos que queremos cambiar de color
node = list(karate.nodes)
labels = [i if i == "Mr Hi" or i == "John A" else ""
                for i in karate.nodes]
# construir un nuevo diccionario de nodos pero esta vez
# con los nombres de sólo los nodos objetivo
nodelabels = dict(zip(node, labels))
# con base en esto creamos una nueva lista de colores
colors = ["yellow" if i == "Mr Hi" or i == "John A" else
"lightskyblue" for i in karate.nodes]
# visualizamos nuestro grafo en forma circular con los nuevos colores
# y el color eje lo podemos cambiar también
nx.draw_circular(karate, labels = nodelabels, node_color = colors,
edge_color = "blue")
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
plt.title('Grafo de interacciones personalizado')
plt.show()
```

Figura 4: Grafo de interacciones con opciones de personalización



Fuente: Elaboración propia

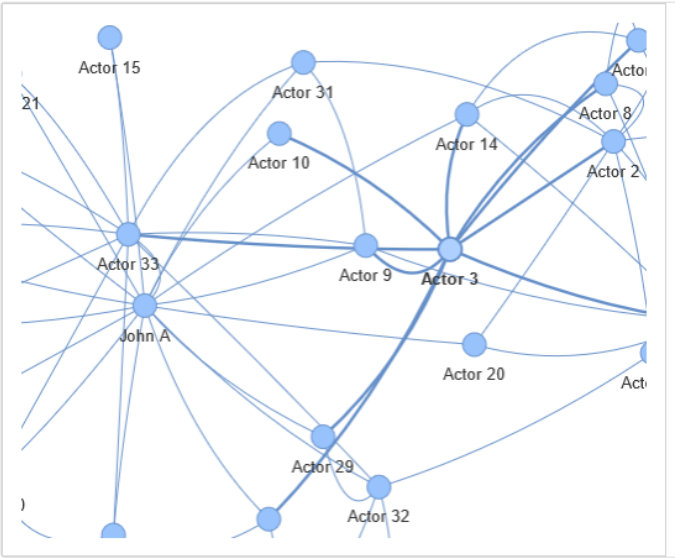
Con NetworkX tenemos más opciones de personalización que dejamos a voluntad del estudiante. Tales opciones incluyen la posibilidad de agregar etiquetas a los ejes y vértices, cambiar tamaños, estilos de aristas, entre otras. Por otro lado, como ya se ha visto las ventajas y las desventajas que hay entre las visualizaciones estáticas y las interactivas. NetworkX a pesar de ser una librería de visualizaciones estáticas permite la integración con librerías de carácter interactivo para grafos exportando la información (GraphViz, D3.js, Bokeh y plotly) o de integración directa como nxviz o Pyviz. Por tal motivo, agregamos un ejemplo usando Pyviz:

```
from pyvis.network import Network
from IPython.display import HTML
# create pyvis Network object
net = Network(height = "500px", width = "600px", notebook = True,
              cdn_resources='remote')  #  remote  si  esta  en  google
colab
net.from_nx(karate) # importar el grafo karate
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

```
# alternativa directa si estamos trabajando en local
#net.show('out1.html', local='in_line', notebook=True)
# para visualizar en colab, guardar el html correctamente
net.write_html(name='out1.html',      local=True,      notebook=True,
open_browser=True)
# y abrir con IPython
with open('out1.html', 'r') as file:
    html_content = file.read()
# visualizar el contenido HTML en google colab
HTML(html_content)
```

Figura 5: Grafo de interacciones interactivo con Pyvis



Fuente: Elaboración propia

Con este ejemplo finalizamos nuestra lectura de profundización, animamos al estudiante a seguir explorando estas dos nuevas herramientas de visualización de grafos y así obtener todo el conocimiento en este tópico especifico de visualización.

Conclusiones

En esta guía de profundización se recalcó la importancia de manejar herramientas adicionales para manejar diferentes tipos de datos, en este caso los

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

grafos, importantes en el análisis y visualización de información. Específicamente, se introdujo a NetworkX como herramienta de visualización estática de grafos pero también se usó Pyvis la cual está integrada directamente con NetworkX y permite una visualización de grafos interactiva. Esperamos que esta lectura haya sido de mucho interés y que haya alcanzado el objetivo final que es proporcionar al estudiante más herramientas en la visualización de datos.

Referencias

Keith McNulty, 2022, Handbook of Graphs and Networks in People Analytics, recuperado el 20 de enero de 2022 de https://ona-book.org/viz-graphs.html#visualizing-graphs-in-python

NetworkX Developers. (n.d.), Tutorial, Recuperado el 20 de enero de 2024 de https://networkx.org/documentation/stable/tutorial.html

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.